Working with API Controller

CC by 4.0

WSO2

Objective

At the end of this module, attendees will get a basic idea of

- What is API Controller?
- Creating an API Project with dev-first approach
- Migrating APIs and API Products to Different Environments
- Migrating Applications to Different Environments
- API Controller Functions

API Controller - Introduction

WSO2

API Controller

- API Controller is a command line tool which can be used for managing APIs, API Products, MCP Servers and Applications.
- API Controller can be used to,
    - Create and import APIs, MCP Servers and API Products
    - Manage Environments
    - Migrate APIs, MCP Servers and API Products among environments. (Dev, Staging, Production etc)
    - Build CI/CD pipeline for APIs
    - Migrate Applications among environments
- API Controller has the ability to work with kubernetes and Choreo Connect
- It also has the ability to perform WSO2 Micro Integrator (WSO2 MI) server specific operations

Creating an API via Dev-first Approach

WSO2

Dev-First Approach of API Creation

- Usually in API Manager, an API is created via Publisher portal.
- APICTL can be used to create an API from scratch or using an existing Swagger or Open API definition
- Dev-First approach introduces a Project based approach, which can be initialized using APICTL.
- In this approach, the project is based on an API definition file (swagger) and other artifacts such as operation policies, API documents, certificates etc can also be associated with the project.
- This approach simplifies the API creation and management and helps automating the process.

Downloading and Initializing the APICTL

- Download API Manager Tooling from https://wso2.com/products/downloads/?product=wso2am&package=tooling based on your preferred platform
- Extract the downloaded archive of the CTL Tool to a desired location
- Navigate to the working directory where the executable CTL Tool resides
- Execute the following command to start the CTL Tool

```
./apictl
```

Creating an API Project Using APICTL

An API Project can be created as follows.

1. As an empty project from scratch

```
apictl init SampleAPI
```

2. With existing API definition file.

```
apictl init Petstore --oas petstore.yaml
```

Executing the above commands will generate an API Project with required artifacts.

API Project Structure

```
├── api.yaml
├── api_meta.yaml
├── deployment_environments.yaml
├── Client-certificates
├── Definitions
│   └── swagger.yaml
├── Docs
├── Endpoint-certificates
├── Image
└── Policies
```

Sub Directory/File	Description
`api.yaml`	The specification of the created API.
`api_meta.yaml`	The meta-information file of the source artifact (This includes the name and the version of the API).
`deployment_environments.yaml`	Specify the gateway environments to which the API should be deployed.
`swagger.yaml`	The Swagger file that is generated when the API is created.
Policies	Contains the operation policies attached to the operations of the API. To learn more about adding a custom operation policy with dynamic data, see Add dynamic data to Operation Policies in an API project.
Client-certificates	Contains the client certificates for Mutual SSL enabled APIs.
Docs	Contains the documents.
Endpoint-certificates	Contains the endpoint certificates for endpoint security enabled APIs.
Image	Contains the thumbnail image of the API.

Changing Default Values

The generated project contains pre-configured set of values. Therefore, before publishing the API, proper values should be provided for mandatory fields in api.yaml file. (apiName, context, production_endpoints and sandbox_endpoints)

Sample configuration:

```yaml
type: api
version: v4.2.0
data:
  id: 7be0e815-47db-4f4f-8ede-3f087c529be0
  name: PizzaShackAPI
  description: This is a simple API for Pizza Shack online pizza delivery store.
  context: /pizzashack
  version: 1.0.0
  provider: admin
  lifeCycleStatus: PUBLISHED
  responseCachingEnabled: false
  cacheTimeout: 300
  hasThumbnail: false
  isDefaultVersion: false
  isRevision: false
  revisionId: 0
  enableSchemaValidation: false
  enableSubscriberVerification: false
  type: HTTP
  transport:
   - http
   - https
  tags:
   - pizza
  policies:
   - Unlimited
  authorizationHeader: Authorization
  securityScheme:
   - oauth2
   - oauth_basic_auth_api_key_mandatory
  visibility: PUBLIC
  visibleRoles: []
  visibleTenants: []
  mediationPolicies: []
  subscriptionAvailability: CURRENT_TENANT
  subscriptionAvailableTenants: []
  additionalProperties: []
  additionalPropertiesMap: {}
```

Changing Environment Specific Parameters

- Parameters which vary from environment to environment such as production and sandbox endpoints etc can be provided via

  an additional parameter file or an

  additional directory containing deployment-related artifacts

- It is recommended to store the parameters file inside the deployment directory if there are certificates included. However, it can be stored somewhere else when certificates are not specified via this parameters file.

Generating Deployment Artifacts Directory

- The following command can be used to generate the deployment-specific directory.

```
apictl gen deployment-dir -s <path-to-source-directory-or-archive>
```

- A project folder with the following default structure will be created in the given directory.

```
DeploymentArtifacts_<API_Name>-<API_Version>
├── api_meta.yaml (api_product_meta.yaml for API Products)
├── certificates
└── params.yaml
```

- You can provide the parameters file using --params flag when importing the API.

```
apictl import api -f dev/PhoneVerification_1.0.zip -e production --params /home/user/custom_params.yaml
```

Configuring Different Endpoint Types

- Different types of endpoints can be specified in the api_params.yaml file
- Following endpoint types are supported.
    - HTTP / Rest endpoints
        - Without load balancing or failover
        - With load balancing
        - With failover
    - HTTP / SOAP endpoints
        - Without load balancing or failover
        - With load balancing
        - With failover
    - Dynamic endpoints
    - AWS Lambda endpoints
        - Using IAM role-supplied temporary AWS credentials
        - Using stored AWS credentials

Adding API Environments

- In order to publish the created API, a running API Manager instance should be configured as an Environment in API Controller
- It is possible to add environments by either manually editing the <USER_HOME>/.wso2apictl/main_config.yaml file or by running below command

```
apictl add env <environment-name> \
        --registration <client-registration-endpoint> \
        --apim <API-Manager-endpoint> \
        --token <token-endpoint> \
        --admin <admin-REST-API-endpoint> \
        --publisher <publisher-portal-endpoint> \
        --devportal <developer-portal-endpoint> \
        --mi <mi-management-endpoint>
```

Required :

(Either)

`--apim` : API Manager endpoint **for** the environments

OR (the following 4)

`--registration` : Registration endpoint **for** the environment

`--admin` : Admin endpoint **for** the environment

`--publisher` : Publisher endpoint **for** the environment

`--devportal` : DevPortal endpoint **for** the environment

Optional :

`--token` : Token endpoint **for** the environment

To add a WSO2 MI :

`--mi` : Management endpoint of the Micro Integrator

Adding API Environment

Example:

All In One

```
apictl add env production \
        --apim https://apim.com:9443
```

Adding a WSO2 MI

```
apictl add env dev \
        --mi https://localhost:9164
```

Distributed

```
apictl add env production \
        --registration https://idp.com:9444 \
        --token https://gw.com:8244/token \
        --admin https://apim.com:9444 \
        --publisher https://apim.com:9444 \
        --devportal https://apps.com:9444
```

Log in to the Environment

- Once the Environment is added, it is required to log in to the environment in order to deploy the API in that environment.
- Logging in to the environment can be done in one of the following ways.

```
apictl login <environment-name> -k
```

```
apictl login <environment-name> -u <username> -k
```

```
apictl login <environment-name> -u <username> -p <password> -k
```

Example

```
apictl login production -u admin -p admin
```

Importing the API to the Environment

- After login, the API can be deployed in the Environment.
- For this the import api command is used. The lifecycle status of the API (Created/ published etc) will be based on the status which is defined in the api.yaml file.

```
apictl import api -f <path to API Project> -e <environment>
```

Example:

```
apictl import api -f SampleAPI -e production
```

Important: By default, only the users with **Internal/devops** role can import the APIs. To support other users, follow these instructions.

Migrating APIs and API Products to Different Environments

- This feature allows to export and import APIs, MCP Servers or API Products between environments without creating them again from scratch.
- Login to the respective API Manager environment before exporting or importing APIs, MCP Servers or API Products.
- It is possible to export a specific API/API Product or export all the APIs belonging to given tenant in either JSON or YAML formatted archives.
- It is possible to provide the revision number when exporting an API/API Product. If not provided, working copy of the API/API Product will be exported.
- If the deployment environments are provided, when importing, a new revision will be created and that revision will be deployed in the given gateway environments. If the deployment environments are not provided, only the working copy will be updated.

Migrating APIs, MCP Servers and API Products to Different Environments Cont.

- Update the params.yaml file if environment specific configurations need to be changed.
- Exported API/API Product/MCP Server archive can be used to either import the API/API Product/MCP Server or seamlessly update an existing API/API Product/MCP Server.

Exporting and Importing APIs

- Export an API

  ```
  apictl export api -n <API-name> -v <version> --rev <revision-number> -r <provider> -e <environment> --preserve-status=<preserve-status> --format <export-format> -k
  ```

- Export all the APIs of a tenant - To export all the revisions and the working copy without considering the deployment status, use --all flag

  ```
  apictl export apis --environment <environment-from-which-artifacts-should-be-exported> --all --format <export-format> --preserve-status --force -k
  ```

- Import an API

  ```
  apictl import api --file <path-to-API-archive> --environment <environment> --preserve-provider=<preserve_provider> --update=<update_api> --skip-cleanup=<skip-cleanup> --params <environment-params-file> --rotate-revision -k
  ```

Importing APIs Cont.

- Promoting APIs in a GIT repository to upper environment via CI/CD

  Set the Source and the Deployment directory paths.

  ```
  apictl set --vcs-source-repo-path <path>
  ```

  ```
  apictl set --vcs-deployment-repo-path <path>
  ```

  Navigate to the Source directory.
  Initialize the git repository with apictl.

  ```
  apictl vcs init
  ```

  Verify the available changes

  ```
  apictl vcs status -e prod
  ```

  If you have a Deployment repository, perform the steps 2 - 4 for that as well.

  Deploy the changes to the upper environment.

  ```
  apictl vcs deploy -e prod
  ```

Exporting and Importing API Products

- Export an API Product

  `apictl export api-product -n <API Product-name> --rev <revision-number> -r <provider> -e <environment> --format <export-format> -k`

- Import API Product

  `apictl import api-product -f <path-to-API-Product-archive> -e <environment> --rotate-revision -k`

- Import API Product with dependent APIs

  `apictl import api-product -f <path-to-API-Product-archive> -e <environment> --rotate-revision -k --import-apis`

- Import API Product to update an existing API preserving the provider

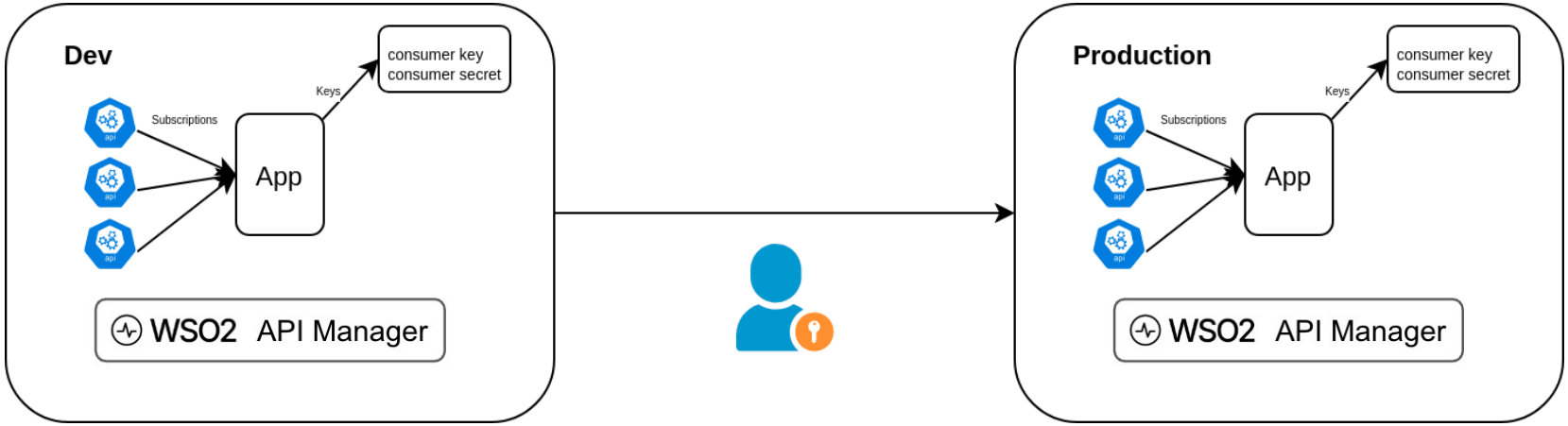
  `apictl import api-product -f <path-to-API-Product-archive> -e <environment> --preserve-provider=<preserve_provider> --update-apis=<update_apis> --skip-cleanup=<skip-cleanup> --rotate-revision -k`

Tip: The same “apictl vcs” commands used for APIs can be used to deploy API Products in a CI/CD flow.

Migrating Applications to Different Environments

WSO2

Migrating Applications to Different Environments



Exporting Applications

- Export an Application using the tool

```
apictl export app -n MyApplication -o admin -e dev
```

This command exports an application named as MyApplication from Owner admin in dev environment. Application will be exported as a ZIP archive.

- Export an Application with OAuth consumer key and secrets

```
apictl export app -n MyApplication -o admin -e dev --with-keys=true
```

admin in dev environment with the consumer key and secret as specified in --with-keys flag.

Importing Applications

- Import Applications to another environment using the tool

```
apictl import app -f MyApplication.zip -e test --update
```

  This will import the MyApplication.zip to the test environment. This will import all the consumer keys, subscribers into the next environment by default. Update flag will update an existing application if already exists in next environment, if not it will create a new one.
  If application already associated with consumer key/secret in the new environment for this application, existing keys in new environment will be not be impacted.

- User can disable importing keys or subscriptions

```
apictl import app -f MyApplication.zip -e test --skip-keys=true --skip-subscriptions=true
```

  Note: By default the applications will be imported under current user executing the command, User can preserve the original user using --preserve-owner flag or change the ownership to a desired owner using --owner flag

Tip: The same “apictl vcs” commands used for APIs can be used to deploy Applications in a CI/CD flow.

Migrating Rate Limiting Policies

WSO2

Managing Rate Limiting Policies

WSO2 API Controller (apictl) allows the following actions on rate limiting Policies.

- Get rate limiting policies in an environment.
- Delete rate limiting policies from an environment.
- Export rate limiting policies from an environment.
- Import rate limiting policies to an environment

```
apictl get policies  rate-limiting  -e prod  -q type:sub
```

```
apictl delete policy rate-limiting -n Gold -e dev --type sub
```

Migrating Common API Policies

WSO2

Migrating Common API Policies

- Following actions on common API Policies are allowed by apictl.
  - List common API Policies in an environment.
  - Export a common API Policy in an environment.
  - Import a common API Policy to an environment.
  - Delete a common API Policy in an environment.

```
apictl get policies api -e dev
```

```
apictl delete policy api -n addHeader -v v1 -e dev
```

API Controller Functions

WSO2

API Controller - Functions Reference

API Controller can also perform these functionalities.

- Environment:
  - Add Environment
  - List Environments
  - Remove Environments
  - Login and Logout from Environments
- APIs and API products
  - Initiating API Projects
  - List APIs/API products in an Environment
  - Filter API listing by providing a query
  - Export API/API product from an Environment
  - Import an API/API Product to an Environment
  - Manage Revisions of APIs/API Products
  - Change the Lifecycle State of an API/API Product
  - Delete API/API product from an Environment
  - API Logging

Apps
- List Applications in an environment
- Delete Application from an Environment
- Export Application from an Environment
- Import Application to an Environment

Throttling Policies and Common Policies
- List Policies in an environment.
- Export Policy in an environment.
- Import a Policy to an environment.
- Delete a Policy in an environment.

Integrating with a Git repository:
- Initialize a git repo with API Controller
- Verify the available changes of APIs/Apps/Products to deploy
- Deploy the changes of APIs/Apps/Products

Let’s try it out!

Creating an API Project Using the API Controller

WSO2